NETWORK INTRUSION DETECTION SYSTEM

Custom Rule-Based Traffic Detection Using Suricata

CodeAlpha Cybersecurity Internship — Task 4: Network Intrusion Detection System

Prepared by: Omoneka Philomena Ekhaizor

Date: October 2026

1. Objective

The objective of this project is to set up a network-based Intrusion Detection System (IDS) capable of monitoring live network traffic and generating alerts for suspicious or malicious activity, using Suricata on a Kali Linux host. Three custom detection rules were written, tested against real traffic (ICMP, a TCP port scan, and an HTTP request), and verified against the engine's alert log to confirm genuine detection rather than assumed functionality.

2. Tools and Environment

- Operating system: Kali Linux, running in Oracle VirtualBox
- IDS engine: Suricata 8.0.7 (open-source network threat detection engine)
- Traffic generation tools: ping (ICMP), nmap (port scanning), curl and Python's built-in http.server (HTTP)
- Target/monitored interface: eth0, target host 10.0.4.1

3. Installation and Configuration

3.1 Installing Suricata

Suricata was installed via the system package manager:

```
sudo apt update sudo apt install suricata -y
```

3.2 Custom detection rules

Three custom rules were created in Suricata's local rules file to detect three distinct categories of network activity: an ICMP ping sweep, a TCP port scan, and an HTTP GET request.

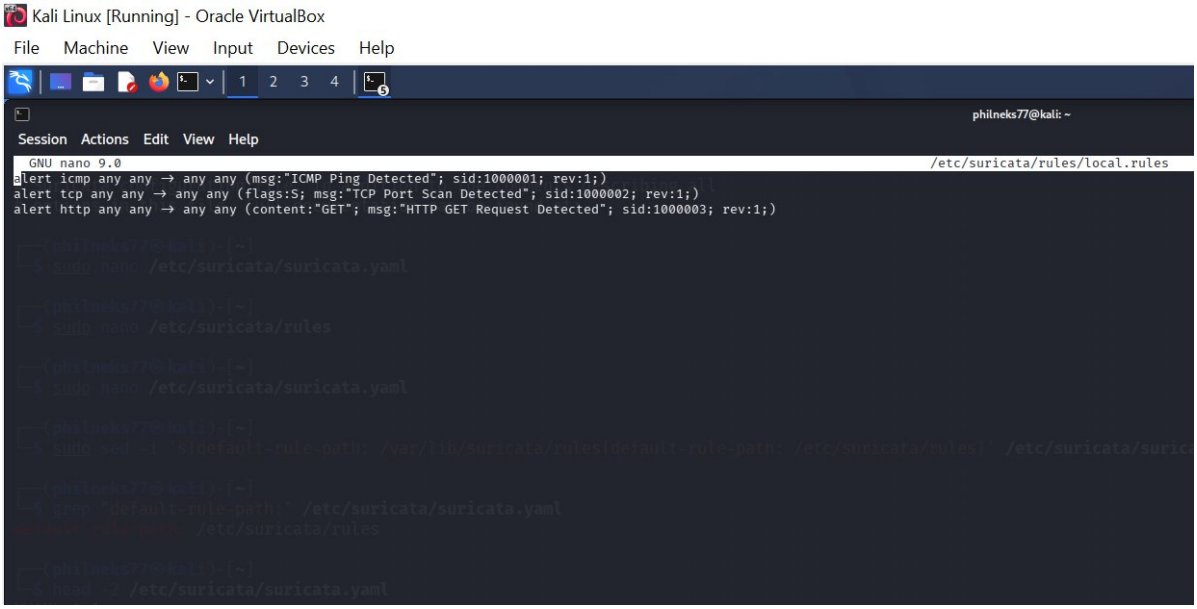


Figure 1. Custom rules defined in /etc/suricata/rules/local.rules.

3.3 Pointing Suricata at the custom rules file

By default, Suricata looks for a rules file that is not present on a fresh installation. The configuration file (suricata.yaml) was edited so that the rule-files directive referenced the custom rules file directly, and the default-rule-path was corrected to point to the directory containing it (/etc/suricata/rules).

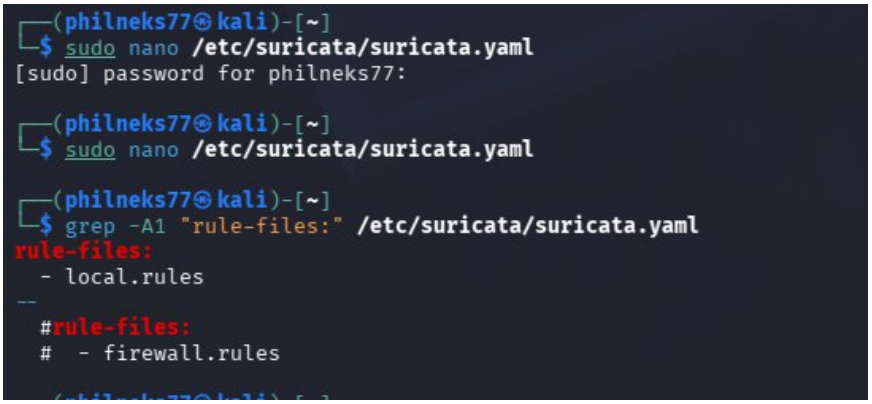


Figure 2. Corrected rule-files directive in suricata.yaml, confirmed via grep.

3.4 Verifying the rules loaded

After correcting the configuration, Suricata was started against the config file and interface, and the log was checked to confirm the rules were parsed and loaded — not merely that the engine started.

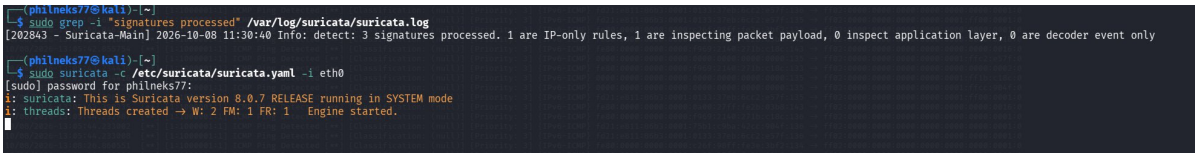


Figure 3. Suricata log confirming 3 signatures processed, and the engine starting cleanly with no rule-loading warnings.

4. Generating Test Traffic

With Suricata running and actively monitoring eth0, three types of traffic were generated against the target host (10.0.4.1) to trigger each custom rule in turn: an ICMP ping, an Nmap port scan of ports 1-100, and an HTTP GET request (served locally with Python's http.server and requested with curl).

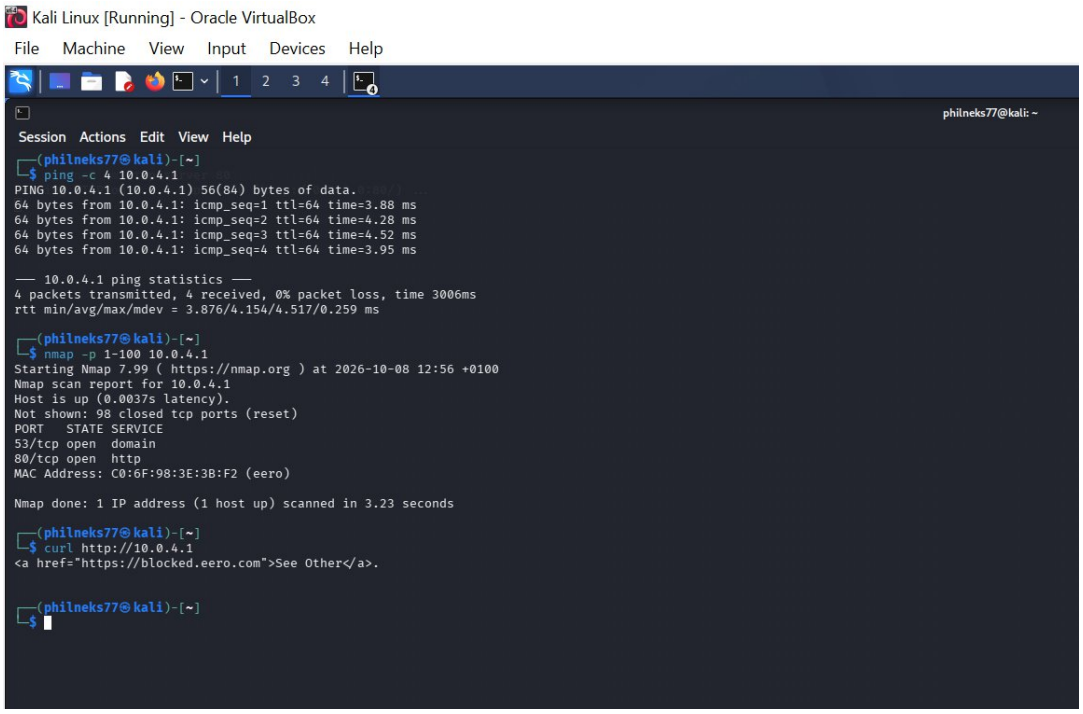


Figure 4. Test traffic generated: ping (ICMP), nmap (TCP port scan), and curl (HTTP GET).

5. Alert Verification

The alert log (/var/log/suricata/fast.log) was monitored live while the test traffic was generated. All three custom rules fired as expected, confirming genuine detection rather than a configuration that merely appeared to run:

- ICMP Ping Detected (sid:1000001) — fired on each ping test
- TCP Port Scan Detected (sid:1000002) — fired once per port probed by Nmap's scan
- HTTP GET Request Detected (sid:1000003) — fired on the curl request to the Python HTTP server

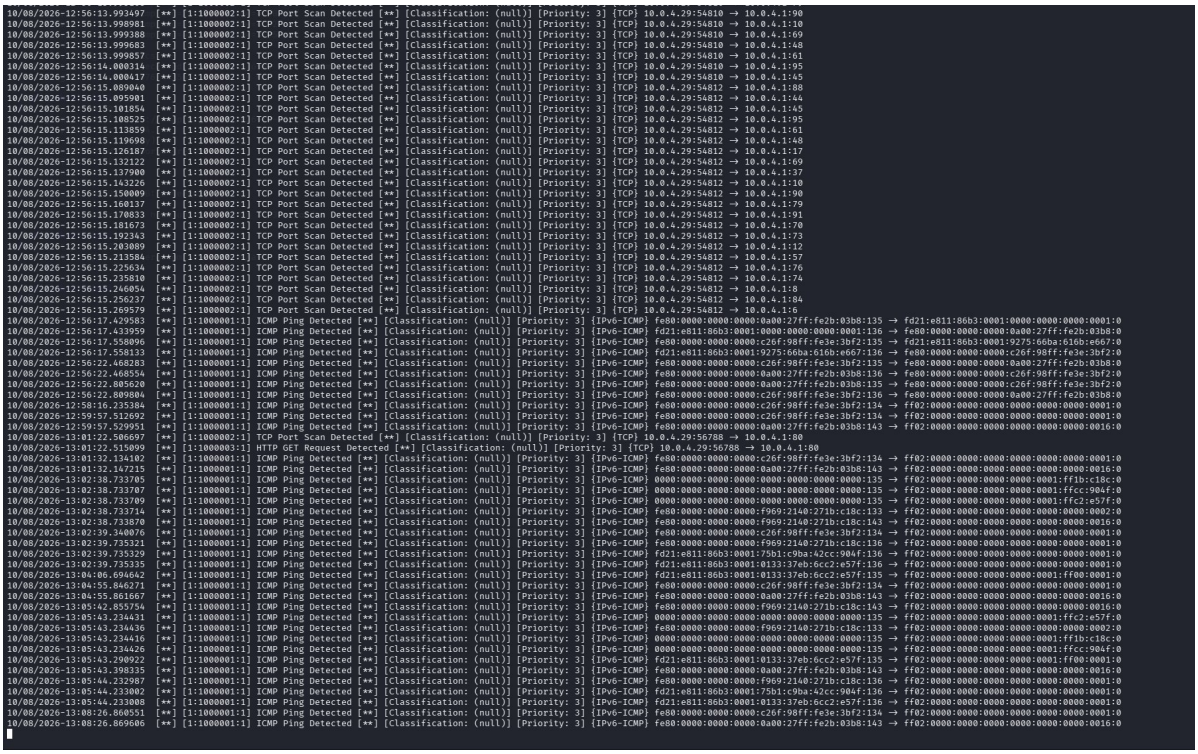


Figure 5. Alert log (fast.log) showing fired alerts for all three rules, including the HTTP GET Request Detected alert at 13:01:22.

6. Findings Summary

Item	Result
IDS engine	Suricata 8.0.7, running in IDS (passive monitoring) mode on interface eth0
Custom rules	3 rules written: ICMP Ping Detected, TCP Port Scan Detected, HTTP GET Request Detected
Configuration issue found	Default rule-files/default-rule-path pointed to a non-existent rules file; corrected to reference the custom local.rules file
Rule load verification	Suricata log confirmed “3 signatures processed” — rules were parsed and active, not just present on disk
ICMP test	Alert fired for every ping sent to the target host
TCP port scan test	Alert fired once per port scanned by Nmap, confirming granular detection across the full scan
HTTP test	Alert fired for the GET request made via curl against the local HTTP server
Conclusion	All three custom detection rules were verified against live, real traffic with matching entries in the alert log — the IDS is functioning as configured

7. Response Mechanisms

Suricata was run in IDS (passive monitoring) mode rather than inline/IPS mode, meaning the engine itself can log an alert but cannot drop live traffic directly. To implement a genuine automated response to detected intrusions, a companion bash script was written to watch the alert log in real time and take action once a source IP crossed a defined threshold of suspicious activity.

7.1 Response script logic

The script (/usr/local/bin/suricata_response.sh) tails the Suricata alert log (fast.log) continuously, watching for TCP port-scan alerts. It maintains a running count of alerts per source IP, and once a source reaches 5 alerts, it automatically adds a DROP rule for that IP using iptables, blocking all further inbound traffic from it. Each action is also written to a dedicated log file (/var/log/suricata_response.log) as a permanent, timestamped record.

7.2 Response triggered by live traffic

With Suricata and the response script both running, a port scan (nmap) was run against the monitored host. The response script counted each port-scan alert as it arrived and triggered the automated block after the fifth alert from the same source IP, exactly as designed.

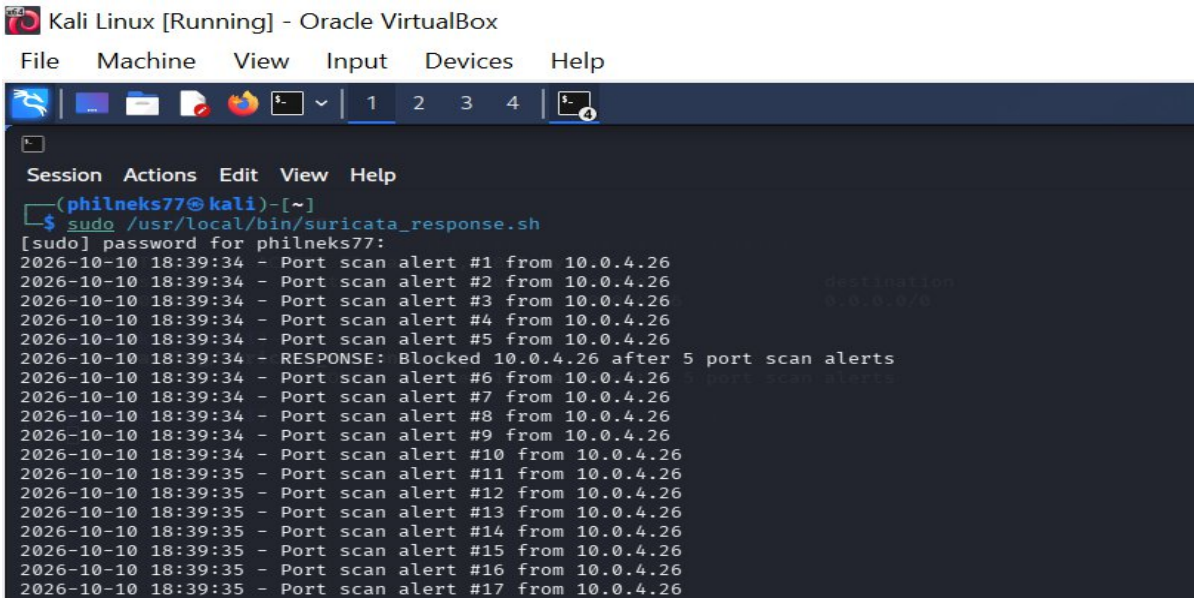


Figure 6. Response script live output: port-scan alerts counted per source IP, triggering an automatic block after the 5-alert threshold was reached.

7.3 Verifying the block was applied

The firewall rule table was checked immediately after the response triggered, confirming that a real DROP rule had been inserted for the offending source IP (10.0.4.26) — not merely logged, but actively enforced at the network level.

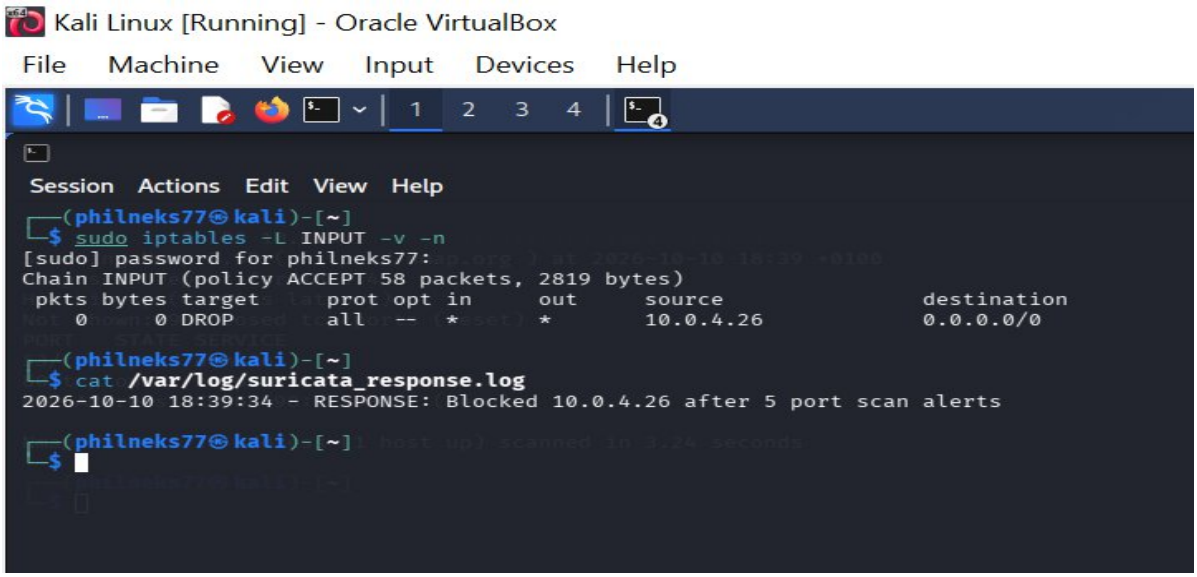


Figure 7. iptables INPUT chain showing the automatically added DROP rule for 10.0.4.26, alongside the permanent response log confirming the block action and timestamp.

7.4 Findings

Item	Result
Response mechanism	Custom bash script monitoring fast.log in real time, triggering an automated iptables block
Trigger threshold	5 TCP port-scan alerts from the same source IP within the monitoring session
Action taken	iptables DROP rule automatically inserted for the offending source IP (10.0.4.26)
Verification	Confirmed via sudo iptables -L INPUT -v -n, showing the live DROP rule
Permanent record	Logged with timestamp to /var/log/suricata_response.log for auditability
Conclusion	The IDS setup was extended with a working, verified response mechanism, satisfying the full detect-and-respond requirement of this task

8. Overall Conclusion

This project delivered a complete, end-to-end intrusion detection and response workflow: Suricata was installed and configured with three custom detection rules, a configuration fault was identified and corrected, detection was verified against live ICMP, TCP port-scan, and HTTP traffic, and a working automated response mechanism was built and verified to block a repeat offending source IP. Every stage

was confirmed against real log output and live command results rather than assumed to work, in keeping with a rigorous, evidence-based approach to network security monitoring.